



Habib University
shaping futures

Habib University

OBJECT-ORIENTED PROGRAMMING FALL 2025

Week 05 - Part 1

Classes and Objects in Detail





THIS WEEK'S AGENDA

- 01. Revisit - Class Members (Member Variables and Functions)
- 02. Revisit - Anatomy of a Class
- 03. Defining Objects
- 04. Constructors
- 05. Constructor Overloading
- 06. Copy Constructor
- 07. Member Functions outside a Class





CLASS MEMBERS





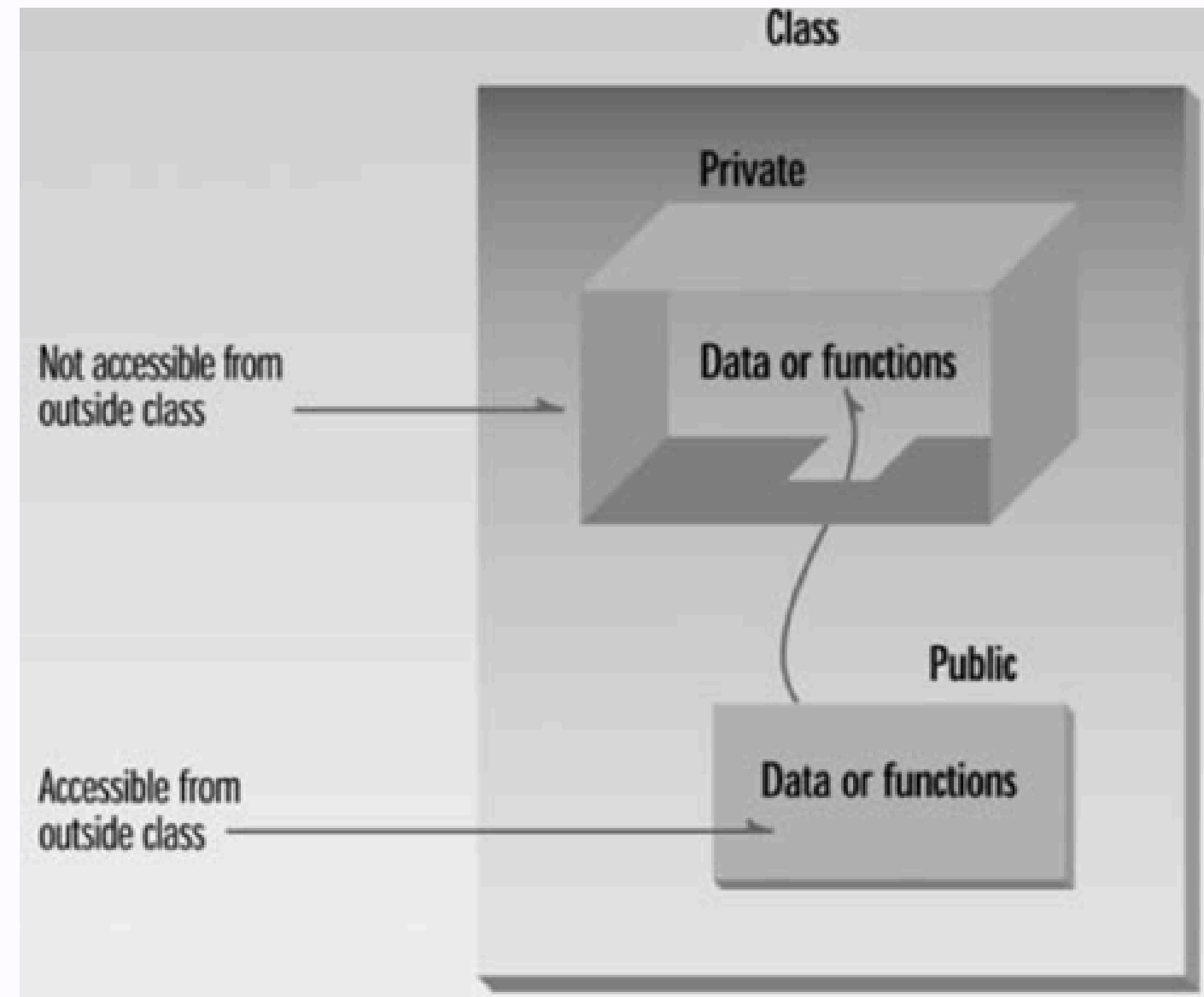
CLASS MEMBERS

- Data items within a class are called **data members** or member data.
- There is no limit to the number of data members that a class can have.
- On the other hand, **member functions** of a class are functions that are included within a class.
 - These are also sometimes called methods of a class.
- Members of a class can either be **private** or **public** courtesy of a key feature of OOP which is **data hiding**.
- Private data is concealed within a class so that it cannot be accessed by functions outside of the class. It can only be accessed from within the class.
- Public data or functions can be accessed from outside the class too.





CLASS MEMBERS





CLASS MEMBERS

- Usually, data within a class is private and the functions are public. This is so that data is hidden and safe from accidental manipulation.
- Functions are usually kept as public so that they can be accessed from outside the class and allow the user to operate on the data of the class.
- However, there is no rule that says data must be private and functions public; in some circumstances you may find you'll need to use private functions and public data.



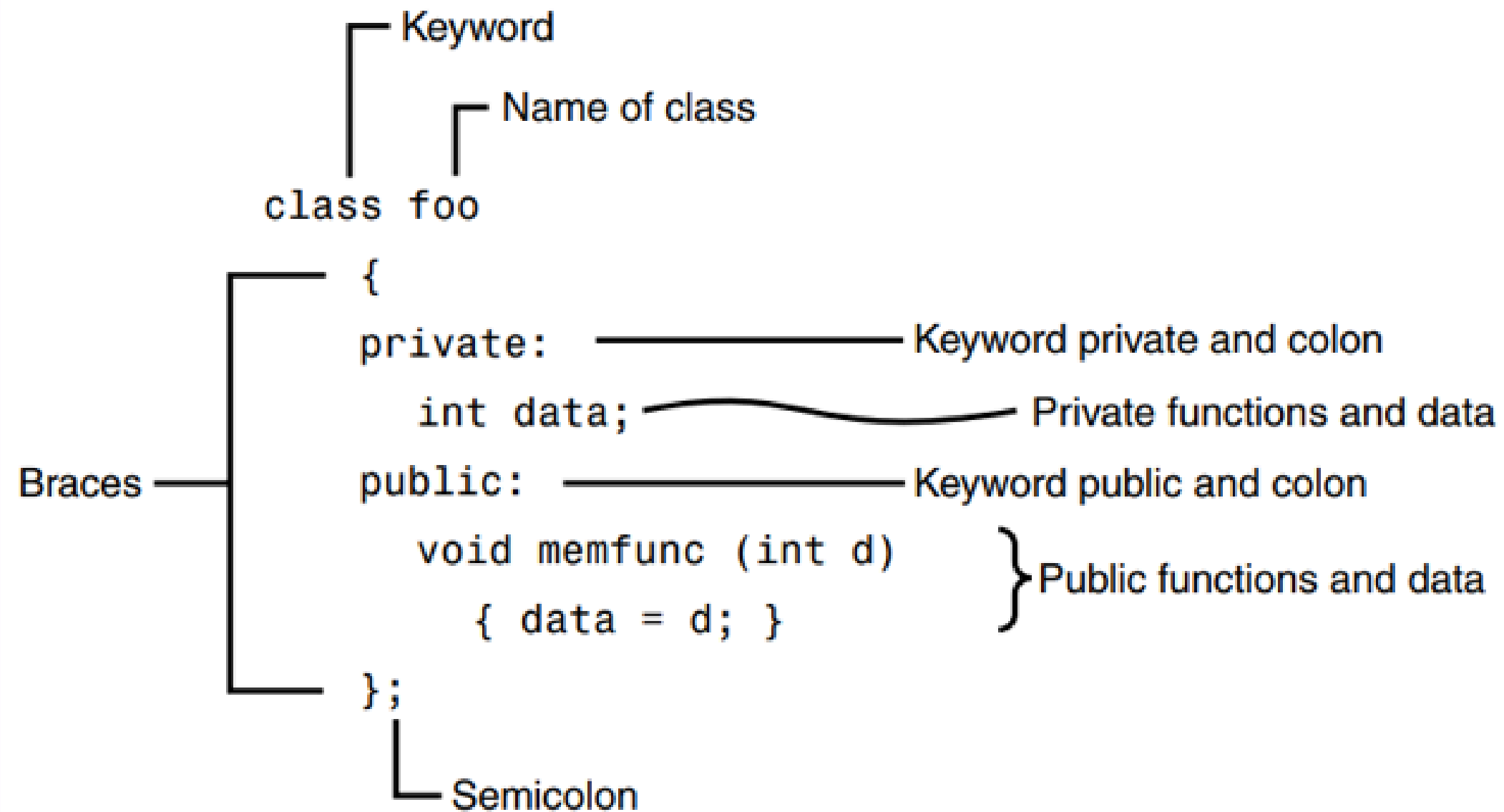


ANATOMY OF A CLASS





ANATOMY OF A CLASS





DEFINING OBJECTS





DEFINING OBJECTS

- Defining an object of a class in C++ is similar to defining a variable of any data type.
- The syntax of defining an object is:
 - `Multiplier mul1;`
 - Where `Multiplier` is the name of the class and `mul1` is the name of the object of the `Multiplier` class.
- When an object is created, it occupies space in the memory.
- Defining them in this way means that we are **instantiating** them, because an **instance of the class** is created (i.e, a specific example of the class).
- Objects are sometimes also called **instance variables**.





DEFINING OBJECTS

- Some object-oriented languages refer to calls to a member function of the class as a message.
- Therefore, the following can be thought of as sending a message to the object `mul1` which tells it to provide us with the multiplication of two numbers:
 - `Multiplier mul1;`
 - `int x, y;`
 - `mul1.multiply(x, y);`





CONSTRUCTORS





CONSTRUCTORS

- A constructor is a special member function that is used to initialize an object of a class.
- The constructor is executed automatically whenever an object is created, we don't need to explicitly call it.
- The constructor is sometimes also called ctor.
- **The name of the constructor has to be the same as that of the class itself.**
- They don't have a return type.





CONSTRUCTORS

- The two different ways of initializing values in constructors are:
 - Initializer List Method
 - Body of Constructor Method

```
Divider(int x, int y) {  
    // std::cout << "Calling function body constructor" << std::endl;  
    a = x;  
    b = y;  
}  
  
// Constructor via parameters assigning using initializer list  
Divider(float x, int y) : a(x), b(y) {  
    // std::cout << "Calling initializer list constructor" << std::endl;  
}
```





INITIALIZER LISTS

- Initializer lists are the preferred way of initializing an object using a constructor.
- There are several reasons why these are preferred such as:
 - When using an initializer list, members initialized are given a value before the constructor even starts to execute.
 - **This is important when we want to initialize const member data.**
- Actions more complicated than simple initialization can be carried out in the constructor body, as with ordinary functions.





OVERLOADING CONSTRUCTORS





OVERLOADING CONSTRUCTORS

- It is possible to overload constructors the same way we overload regular functions.
- These are useful when we want to initialize objects with a variety of initial values.
- Such constructors have the same name (**recall from when we did overloaded functions**), but different signature (**recall what constitutes a function signature**).
- Lets look at an example:





OVERLOADING CONSTRUCTORS

```
1  #include <iostream>
2
3  class Divider {
4      private:
5          int a, b;
6      public:
7          // Constructor with 0 args
8          Divider() : a(0), b(1) {}
9          // Constructor with 2 args
10         Divider(int x, int y) : a(x), b(y) {}
11 };
12
13
14 int main() {
15     Divider d1(10, 2);
16     Divider d2();
17     return 0;
18 }
```





COPY CONSTRUCTORS





COPY CONSTRUCTORS

- Another way of initializing an object is to initialize it with another object of the same type.
- C++ allows you to do this by default, you do not need to write extra code for this on your own.
- **The default copy constructor is already built into all classes.**

```
27     int main() {  
28         Divider d1(10, 2); // two-arg constructor  
29         Divider d2(d1); // one-arg constructor  
30         Divider d3 = d1; // also one-arg constructor  
31         return 0;  
32     }
```





COPY CONSTRUCTORS

- This causes the default copy constructor for the Distance class to perform a **member-by-member copy** of d1 into d2.
- A different format that has exactly the same effect is when we do the following:
 - `Divider d3 = d1;`
- This causes **d1** to be copied **member-by-member** into **d3**.





COPY CONSTRUCTORS

- In C++, copy constructors usually make a **shallow copy** by default, which means that it copies the values of member variables directly.
- If the class contains pointers or dynamically allocated memory, the default copy construct will copy the pointer values.
- This leads to both the original and the copied objects to point to the same memory which can cause issues like double deletion or unintended and unwanted side effects.
- To make a deep copy (where the copy constructor allocates new memory and copies the content of the original object's pointed-to data) we need to define our own copy constructor.





COPY CONSTRUCTORS

```
3  class Divider {
4      private:
5          int *a, *b;
6      public:
7          // Default constructor
8          Divider(int x, int y) {
9              a = new int(x);
10             b = new int(y);
11         }
12         // Custom copy constructor
13         Divider(const Divider &d) {
14             a = new int(*(d.a));
15             b = new int(*(d.b));
16         }
17         // Destructor
18         ~Divider() {
19             delete a;
20             delete b;
21         }
22     };
```





MEMBER FUNCTIONS OUTSIDE A CLASS





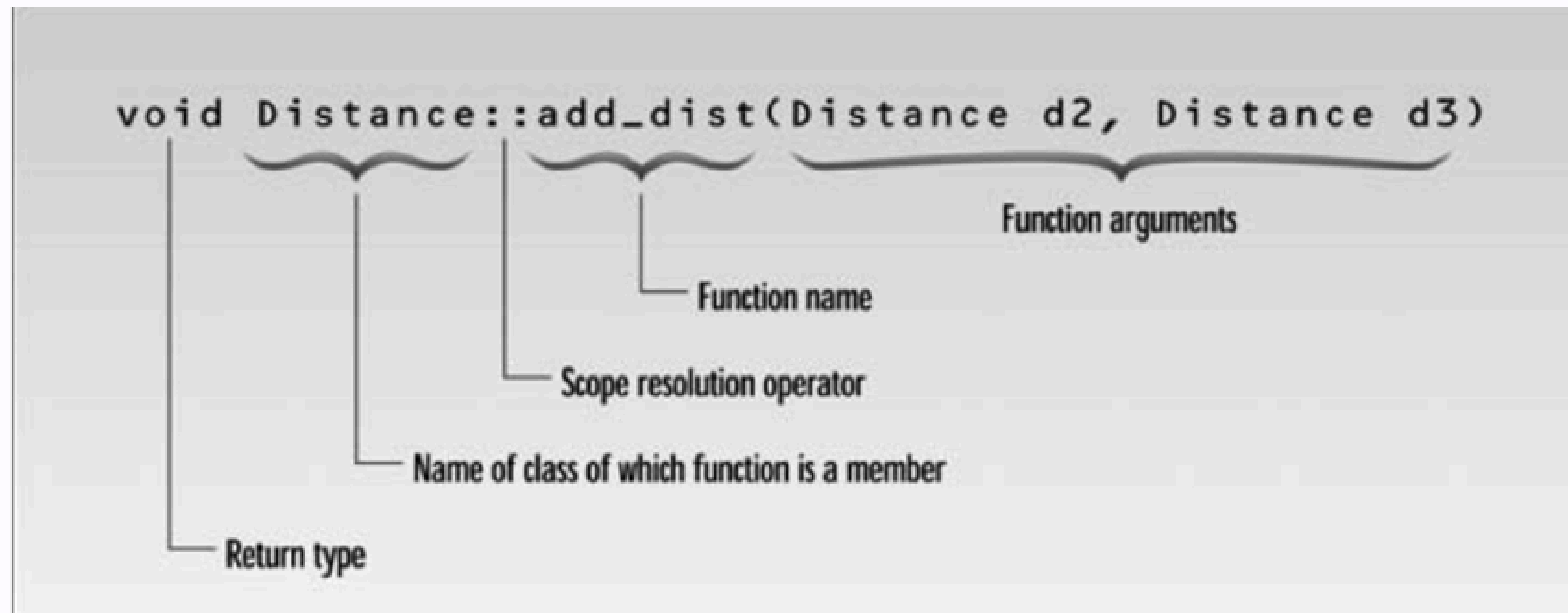
MEMBER FUNCTIONS

- In C++ member functions can be defined outside of the class as well.
- But first, we need to declare that function within the class. For example:
 - `void divide(int, int); // declaration`
- The function can then be defined outside the class in the following way:
 - `void Divider::divide(int x, int y); // definition`
- Let's look at the syntax of this more closely, and then we will do a demo.





MEMBER FUNCTIONS





SUMMARY





SUMMARY

- This session was a bit of an in-depth look at Object Oriented Programming.
- A class can be considered as an extension of a structure as it contains both data and functions on that data.
- Access modifiers such as **public** and **private** help control the access of both the data and the functions within a class.





READINGS

- Object-Oriented Programming in C++ by Robert Lafore - Chapter 06





ACKNOWLEDGEMENTS

Many of the slides of this lecture have been taken/modified from the lecture notes of:

- Object Oriented Programming Fall 2024 by Dr. Faisal Alvi, Assistant Professor, Computer Science
- Object Oriented Programming Fall 2023 by Dr. Musabbir Majeed, Assistant Professor, Computer Science

